# REOAGMS team guide

## 1. Is the demo data real?

Yes. Nothing is hard-coded in the UI. The **simulator** ( `scripts/simulator/simulator.mjs` ) logs in as the administrator and calls the same REST APIs the console uses, through the gateway. Every record is validated, written to the owning service's PostgreSQL database, and triggers the normal side effects (audit log, RabbitMQ events, alerts, work orders, notifications).

Row counts in the running stack (query any time):

```
 docker exec reoagms-asset-db      psql -U asset_user      -d asset_db      -c "select (select
count(*) from facilities) f, (select count(*) from assets) a, (select count(*) from sensors)
s"
docker exec reoagms-monitoring-db psql -U monitoring_user -d monitoring_db -c "select
count(*) from sensor_readings"
```

| Database | Tables seeded |
|---|---|
| identity_db | users (5), roles, audit_logs |
| asset_db | facilities (5), assets (11), sensors (32) |
| monitoring_db | alert_rules (7), sensor_readings (16,000+), telemetry, alerts |
| maintenance_db | technicians (4), work_orders, maintenance_tasks, maintenance_history |
| notification_db | notifications, email_templates (1) |
| analytics_db | kpis (6), dashboards (4), energy_reports (5), performance_reports (5) |

Not seeded: `sms_notifications` (needs an SMS gateway).

## 2. The seeder

- **Automatic:** the `simulator` container seeds on first start (idempotent; every stage checks before inserting), back-fills 24 h of history, then streams live telemetry and injects a fault about once a minute.
- **Seed only, no live stream:** `.\scripts\seed-database.ps1`
- **Wipe and re-seed (destructive):** `.\scripts\seed-database.ps1 -Reset`
- Demo user accounts are created by the Identity Service itself ( `DemoUserInitializer` ) and can be disabled with `REOAGMS_SEED_DEMO_USERS=false` .

## 3. Run it

```
 Copy-Item .env.example .env -ErrorAction SilentlyContinue
docker compose up --build -d      # first run: several minutes
docker compose ps                 # 19 containers
```

Open http://localhost:5173. If a host port is taken (3000, 5433, 6379 are on this laptop), set `GRAFANA_PORT` in `.env`; Redis has no host port; identity-db can be remapped in a git-ignored `docker-compose.override.yml`.

## 4. Credentials

| System | URL | Login |
|---|---|---|
| Console | http://localhost:5173 | `admin@reoagms.com` (Administrator), `manager@`, `operator@`, `engineer@`, `executive@reoagms.com`; password **`Reoagms@2026`** |
| Grafana | http://localhost:3001 (default 3000 via `GRAFANA_PORT`) | **`admin / reoagms`** |
| RabbitMQ UI | http://localhost:15672 | `reoagms` / `reoagms_password` |
| Prometheus | http://localhost:9090 | none |

## 5. Who owns what, and what to expect to be asked

| Member | Owns | Likely question → short answer |
|---|---|---|
| **Charith** | Architecture, Identity, Gateway, React console, Docker, RabbitMQ, Kubernetes, integration | *Why microservices?* Independent deployment, fault isolation, and clear business domains (report §3.4). *Where is RBAC enforced?* Gateway `RolePolicy` plus Spring Security in Identity. |
| **Nemal** | Asset Service | *How is search done?* JPA Specifications with paging (default 20). *Where are images stored?* Per asset via the image controller. |
| **Govindu** | Monitoring Service | *How do you avoid alert floods?* One OPEN alert per rule and asset. *What if Redis dies?* Cache errors are swallowed and the database answers. |
| **Thurunu** | Maintenance Service | *How are orders created automatically?* Consumer on `maintenance.alert-opened.v1` for HIGH/CRITICAL, skipping assets with an active corrective order. *Honest gap:* only 3 unit tests so far. |
| **Yasiru** | Notification Service | *Why FAILED emails?* No SMTP credentials; reason and retries are stored, scheduler retries up to 3 times. *SMS?* Dispatcher exists; no gateway configured. |
| **Sachinthana** | Analytics Service | *How is energy computed?* Average power per sensor per hour × hours; only POWER readings; variance vs rated × time. |
| **All** | Tests, docs, validation, errors | *Test status?* 60 test methods; Monitoring, Maintenance, Notification and Analytics unit suites pass; Asset suite not re-run recently. |

## 6. Known gaps (say them before the examiner does)

- OpenAPI/Postman exist for Monitoring and Analytics only.

- Maintenance and Identity have very few tests; no automated end-to-end test (verified by script and manually).
- Email requires SMTP credentials; SMS has no provider.
- Alert and notification lists are not paginated in the API.
- Secrets and demo accounts are development defaults.

## 7. Material

- 10-minute live script: `deliverables/DEMO_SCRIPT.pdf`
- Slides: one team deck `deliverables/REOAGMS-Team-Presentation.pptx` (10 slides, dark modern theme, timed to the 10-minute demo; speaker notes hold the exact words) — regenerate with `node scripts/presentations/build-decks.js` (needs `pptxgenjs`).
- Contracts and schemas: `docs/INTEGRATION_CONTRACTS.md`, `docs/api`, `docs/database`, `docs/diagrams`.